เอกสารออกแบบโครงงาน

KUFT/2.0

การรับส่งไฟล์แบบเข้ารหัสระหว่างคอมพิวเตอร์สองเครื่อง

Kasetsart University File Transfer, version 2

โปรแกรมรับส่งไฟล์แบบ client/server ที่ออกแบบโปรโตคอลชั้นแอปพลิเคชันขึ้นเอง ทำงานบน TCP ร่วมกับ TLS 1.3 พร้อมกลไกจับคู่ที่ผูกการพิสูจน์ตัวตนเข้ากับ ใบรับรองของช่องทางสื่อสาร

วิชา CS-01418351 Computer Networks

งาน Project 1: Socket Programming

ภาษา Python 3.10+ · ประมาณ 4,000 บรรทัด · 6 โมดูล

สารบัญ

1. โปรแกรมนี้คืออะไร และใช้สำหรับทำอะไร

1.1 ปัญหาที่ต้องการแก้

1.2 ตัวโปรแกรมประกอบด้วยอะไรบ้าง

1.3 เป้าหมายการออกแบบ

1.4 สิ่งที่จงใจไม่ทำ

2. คุณลักษณะของแอปพลิเคชัน (Characteristics)

2.1 สรุปคุณลักษณะเชิงเครือข่าย

2.2 สถาปัตยกรรมและโมดูล

2.3 โมเดลการทำงานพร้อมกัน

2.4 ลักษณะของทราฟฟิกและการใช้ทรัพยากร

2.5 แบบจำลองภัยคุกคาม

3. Service Model ของ Transport Layer: เลือก TCP เพราะอะไร

3.1 ความต้องการที่แท้จริงของแอปพลิเคชัน

3.2 Service model สองแบบที่มีให้เลือก

3.3 การจับคู่ความต้องการกับกลไกของ TCP

3.4 เหตุผลชี้ขาด: TLS 1.3 ทำงานบน TCP

3.5 ช่องว่างใน service model ของ TCP ที่ต้องเติมเอง

3.6 สิ่งที่ TCP ไม่รับประกัน แต่เราไม่ต้องการ

3.7 กรณีที่ UDP จะเป็นคำตอบที่ถูกต้อง

4. การออกแบบ Application-Layer Protocol

4.1 ข้อจำกัดที่การออกแบบต้องทำให้ได้

4.2 รูปแบบข้อความ

4.3 Request line และ Status line

4.4 การตัดสินใจสี่ข้อเรื่อง framing

4.5 ชีดจำกัดที่บังคับใช้

4.6 ชุดคำสั่ง (Methods)

4.7 เฮดเดอร์ทั้งหมด

4.8 รหัสสถานะ (Status codes)

4.9 Request และ Response messages ทุกคำสั่ง

4.10 กลไกจับคู่ (Pairing exchange)

4.11 ความหมายของการโอนย้าย

4.12 State machine ของ session

4.13 การกักขอบเขต (Jail)

5. สรุปคำตอบ

1. โปรแกรมนี้คืออะไร และใช้สำหรับทำอะไร

คำตอบโดยย่อ

KUFT/2.0 คือโปรแกรมรับส่งไฟล์แบบ client/server สำหรับย้ายไฟล์ **โดยตรง** ระหว่างคอมพิวเตอร์สองเครื่องที่อยู่บนเครือข่ายเดียวกัน โดยข้อมูลถูกเข้ารหัสตลอดเส้นทาง ทั้งสองฝั่งพิสูจน์ตัวตนซึ่งกันและกัน และไฟล์ที่รับมาถูกตรวจสอบความถูกต้องด้วย SHA-256 ก่อนถูกส่งมอบให้ผู้ใช้ ไม่ผ่านบุคคลที่สาม ไม่ต้องมีบัญชีผู้ใช้ และไม่ต้องตั้งค่าอะไรนอกจากรหัสจับคู่หนึ่งรหัส

1.1 ปัญหาที่ต้องการแก้

คอมพิวเตอร์สองเครื่องอยู่บนเครือข่ายเดียวกัน — Wi-Fi ที่บ้าน โต๊ะแล็บ หรือ hotspot จากมือถือ — เครื่องหนึ่งมีไฟล์ที่อีกเครื่องต้องการ วิธีที่คนทั่วไปใช้กันนั้น มีข้อเสียชัดเจนทุกวิธี

วิธีที่ใช้กันทั่วไป	ปัญหาที่แท้จริง
ส่งอีเมลหาตัวเอง	จำกัดขนาดไฟล์ และไฟล์ต้องผ่านเซิร์ฟเวอร์ของบุคคลที่สาม
Cloud drive / แอปแชต	อัปโหลดขึ้นอินเทอร์เน็ตแล้วดาวน์โหลดกลับลงมา เพียงเพื่อย้ายไฟล์ในระยะสองเมตร ต้องมีบัญชีและต้องมีเน็ตที่ใช้งานได้
USB flash drive	ต้องเดินไปเสียบเอง ช้า และตัวแฟลชไดรฟ์เองก็เป็นช่องทางแพร่มัลแวร์
Windows file sharing (SMB)	ใหญ่เทอะทะ ต้องตั้งค่า domain/workgroup และเป็นบริการที่ทุกคู่มือความปลอดภัยบอกให้ปิดไฟร์วอลล์กั้นไว้
`ftp` / `python -m http.server`	ส่งเป็น plaintext ใครก็ตามที่อยู่บน Wi-Fi เดียวกันอ่านไฟล์ได้ และถ้าเป็น FTP ก็อ่านรหัสผ่านได้ด้วย

ทุกวิธีที่ว่ามา **ไม่อ้อม ก็ไม่เข้ารหัส หรือไม่ก็ทั้งสองอย่าง** KUFT จึงเกิดขึ้นเป็นคำตอบที่เล็กและตรงไปตรงมา: พิมพ์คำสั่งเดียวในแต่ละเครื่อง ไฟล์วิ่งตรงระหว่างสองเครื่องผ่านเครือข่ายภายใน

1.2 ตัวโปรแกรมประกอบด้วยอะไรบ้าง

- **โปรโตคอลชั้น Application** ชื่อ KUFT/2.0 ซึ่งออกแบบขึ้นเองทั้งหมด และกำหนดเป็นเอกสารข้อกำหนดแยกต่างหาก
- **เซิร์ฟเวอร์** ที่แชร์ไดเรกทอรีเดียว และไม่ให้เข้าถึงอะไรที่อยู่เหนือขึ้นไป
- **ไคลเอนต์** ที่มี shell แบบโต้ตอบ (`ls` , `cd` , `get` , `put` , `rm` , `mv` , `rn` , `mkdir` , `stat`) และโหมดคำสั่งเดียวจบสำหรับเขียนสคริปต์
- **กลไกการจับคู่ (pairing)** ที่ให้การพิสูจน์ตัวตนแบบสองทาง โดยไม่ต้องพึ่ง certificate authority, directory service หรือบัญชีคลาวด์ใด ๆ

การใช้งานจริงมีเพียงสองคำสั่ง:

```
เครื่องที่มีไฟล์       python kuft.py serve
                          → พิมพ์ที่อยู่ IP และรหัสจับคู่ออกมาบนหน้าจอ

เครื่องที่ต้องการไฟล์  python kuft.py connect 192.168.1.114
                          → พิมพ์รหัสจับคู่หนึ่งครั้ง แล้วใช้ ls / get / put ได้ทันที
```

1.3 เป้าหมายการออกแบบ

เรียงตามลำดับความสำคัญ เมื่อเป้าหมายขัดกัน ข้อที่อยู่บนกว่าชนะเสมอ

1. **ถูกต้อง** — ไฟล์ที่ไปถึงต้องเหมือนไฟล์ต้นทางทุกไบต์ ไม่เช่นนั้นการโอนย้ายต้องล้มเหลวอย่างเห็นได้ชัด ไม่มีเส้นทางที่ข้อมูลเสียหายแบบเงียบ ๆ
2. **เป็นความลับและพิสูจน์ตัวตนได้** — ผู้โจมตีที่อยู่บน Wi-Fi เดียวกัน ต้องอ่านข้อมูลไม่ได้ และปลอมเป็นฝั่งใดฝั่งหนึ่งไม่ได้
3. **เล็กพอที่จะเข้าใจได้ทั้งหมด** — รวมทั้งโครงงานไม่ถึง 4,000 บรรทัด รูปแบบข้อมูลบนสายพอดีหนึ่งหน้ากระดาษ และอ่านได้จาก packet capture โดยตรง
4. **ใช้งานได้ด้วยคำสั่งสองคำสั่ง** — ถ้าต้องอ่านคู่มือก่อนใช้ ก็ถือว่าล้มเหลวตามโจทย์ที่ตั้งไว้เอง
5. **ใช้ทรัพยากรจำกัด** — โอนไฟล์ 40 GB ต้องใช้หน่วยความจำเท่ากับโอนไฟล์ 4 KB

1.4 สิ่งที่จงใจไม่ทำ

- **ไม่มีบัญชีผู้ใช้รายบุคคล** — มีรหัสจับคู่ร่วมรหัสเดียว ระดับความไว้วางใจเดียว เหมาะกับเครื่องสองเครื่องของคนคนเดียว ไม่เหมาะกับห้องแล็บหลายผู้ใช้
- **ไม่มี certificate authority** — ใบรับรองเป็นแบบ self-signed และใช้การปักหมุด (pinning) แบบเดียวกับ SSH แทน
- **ไม่มีการโอนแบบวนลึกทั้งไดเรกทอรี** — การลบและการโอนแบบวนลึก ควรเป็นลูปฝั่งไคลเอนต์ที่ผู้ใช้มองเห็นทุกขั้นตอน
- **ไม่มี service discovery** — ต้องพิมพ์หมายเลข IP เอง

2. คุณลักษณะของแอปพลิเคชัน (Characteristics)

2.1 สรุปคุณลักษณะเชิงเครือข่าย

ตารางนี้คือคุณลักษณะที่กำหนดว่าโปรแกรมนี้ต้องการอะไรจากชั้น Transport ซึ่งจะถูกใช้เป็นฐานของการตัดสินใจในหัวข้อที่ 3

คุณลักษณะ	รายละเอียด
สถาปัตยกรรม	Client/Server — ฝั่งหนึ่งรัน `serve` เพื่อแชร์ไดเรกทอรี อีกฝั่งรัน `connect` ไม่ใช่ peer-to-peer
ทิศทางข้อมูล	สองทาง — ดาวน์โหลด (`GET`) และอัปโหลด (`PUT`) บน connection เดียวกัน
ลักษณะ session	**มีสถานะ (stateful)** และอายุยาว — ทักทาย → พิสูจน์ตัวตน → ทำงานหลายคำสั่งติดกัน → ปิด
ขนาดข้อมูล	ไม่จำกัดโดยธรรมชาติ — ตั้งแต่ไฟล์ศูนย์ไบต์ถึงหลายสิบกิกะไบต์ในหนึ่งข้อความ
ความทนต่อการสูญหาย	**ศูนย์** — ไฟล์ที่ขาดไปหนึ่งไบต์คือไฟล์เสีย ไม่ใช่ไฟล์ที่แย่ลงนิดหน่อย
ความทนต่อความหน่วง	**สูงมาก** — ไม่ใช่งานเรียลไทม์ ไบต์ที่มาช้ามีค่าเท่ากับไบต์ที่มาตรงเวลา
ความต้องการ throughput	ยิ่งมากยิ่งดี แต่**ไม่มีขั้นต่ำที่ต้องรับประกัน** การโอนที่ช้ากว่ายังคงถูกต้องสมบูรณ์
ความปลอดภัย	จำเป็น — ความลับ ความถูกต้อง และการพิสูจน์ตัวตนสองทาง
จำนวนผู้ใช้พร้อมกัน	น้อย — ค่าเริ่มต้น 8 connection พร้อมกัน
ขอบเขตเครือข่าย	LAN เดียวกันเป็นหลัก (ขยายข้ามระยะไกลได้ผ่าน VPN, port forward หรือ tunnel)

2.2 สถาปัตยกรรมและโมดูล

```
kuft.py                 จุดเข้าเพียงจุดเดียว  กระจายไปยังคำสั่งย่อย
src/kuft/
    bootstrap.py        หา Python ที่มี cryptography ติดตั้งอยู่
    crypto.py           กุญแจ ใบรับรอง รหัสจับคู่ known hosts
    protocol.py         รูปแบบข้อมูลบนสาย: encode, decode, stream, status codes
    server.py           TLS listener, session state machine, request handlers
    client.py           การจับคู่ การปักหมุด การโอนย้าย และ shell แบบโต้ตอบ
    term.py             สี แถบความคืบหน้า ตาราง การรับ input แบบซ่อน
```

โมดูล	บรรทัด	หน้าที่
`protocol.py`	366	รูปแบบข้อมูลบนสาย และเฉพาะรูปแบบข้อมูลบนสายเท่านั้น ไม่รู้จักไฟล์ socket TLS หรือผู้ใช้เลย
`crypto.py`	379	ตัวตนและความไว้วางใจ: สร้างใบรับรอง, PBKDF2, การสร้าง proof และ `known_hosts.json`
`server.py`	693	`Jail` (กักขอบเขต path), `RateLimiter` , `Session` (state machine), `serve()` (accept loop)
`client.py`	722	นโยบายการปักหมุด, `download()` / `upload()` และ `Shell`
`term.py`	360	ทุกไบต์ที่แสดงต่อผู้ใช้ มีที่เดียวที่ต้องรองรับ `--no-color`
`bootstrap.py`	287	รีสตาร์ตเข้าสู่ Python ที่ `import cryptography` ได้

> **การตัดสินใจเชิงโครงสร้างที่สำคัญที่สุด**
>
> `protocol.py` **ถูก import โดยทั้งไคลเอนต์และเซิร์ฟเวอร์** ทั้งโปรเจกต์จึงมี encoder ตัวเดียวและ decoder ตัวเดียวเท่านั้น นั่นแปลว่าสองฝั่ง เข้าใจรูปแบบข้อมูลไม่ตรงกัน *ไม่ได้เลยในเชิงกายภาพ* ถ้าแก้ framing ทั้งสองฝั่งจะพังพร้อมกันทันทีในชุดทดสอบ

2.3 โมเดลการทำงานพร้อมกัน



- **หนึ่ง thread ต่อหนึ่ง connection** — เหมาะกับงานนี้เพราะ session มีอายุยาว มีสถานะ และถูกครอบงำด้วย blocking I/O บน socket และไฟล์ การใช้ event loop ไม่ได้ประโยชน์อะไรที่ 8 ไคลเอนต์พร้อมกัน แต่แลกมาด้วย ความซับซ้อนของโค้ดอย่างมาก
- `threading.Semaphore` **จำกัดจำนวนพร้อมกัน** ที่ `--max-clients` (ค่าเริ่มต้น 8) ไคลเอนต์ที่เกินจะถูกปฏิเสธด้วย `503 SERVICE UNAVAILABLE` — และคำปฏิเสธนั้นถูกส่ง*ภายใน TLS* จึงเป็นการตอบตามโปรโตคอลจริง ไม่ใช่การตัดการเชื่อมต่อทิ้งเฉย ๆ
- `RateLimiter` **ต่อ connection** (fixed window, ปิดไว้เป็นค่าเริ่มต้น) ตอบ `429 TOO MANY REQUESTS` พร้อมเฮดเดอร์ `retry-after`
- **socket มี timeout 300 วินาที** ไคลเอนต์ที่หายไปกลางคันจึงคืนช่องว่างให้ระบบ
- **accept loop ใช้การวนถามด้วย timeout 0.5 วินาที แทนการบล็อก** — เป็นการแก้ปัญหาเฉพาะของ Windows ที่ไม่ขัดจังหวะ blocking socket call เมื่อกด Ctrl+C

2.4 ลักษณะของทราฟฟิกและการใช้ทรัพยากร

- **ทราฟฟิกไม่สมมาตรอย่างรุนแรง** — ข้อความควบคุมมีขนาดหลักร้อยไบต์ แต่ body ของการโอนไฟล์มีขนาดหลายกิกะไบต์ การออกแบบจึงต้องแยกสองสิ่งนี้ออกจากกัน
- **สตรีมเป็นบล็อกขนาดคงที่ 256 KiB** ข้อมูลไม่เคยถูกโหลดเข้าหน่วยความจำทั้งไฟล์ และค่า hash ถูกคำนวณไปพร้อมกับการสตรีม จึงไม่ต้องอ่านข้อมูลซ้ำอีกรอบเพื่อตรวจสอบ
- **หน่วยความจำคงที่** — ไฟล์ 40 GB ใช้หน่วยความจำเท่ากับไฟล์ 4 KB
- **หลายคำสั่งต่อหนึ่ง connection** — TLS handshake และการจับคู่เกิดขึ้นครั้งเดียว ต้นทุนนั้นถูกกระจายไปทั่วทั้ง session

2.5 แบบจำลองภัยคุกคาม

ผู้โจมตีที่โปรแกรมนี้ป้องกัน คือ**เครื่องอื่นบนเครือข่าย Wi-Fi เดียวกัน** ที่สามารถดักฟังทราฟฟิกทั้งหมด และสามารถแทรกตัวอยู่กลางทางเพื่อปลอมเป็นเซิร์ฟเวอร์ได้ แต่ *ไม่* รู้รหัสจับคู่ที่ถูกอ่านออกจากหน้าจอ

ป้องกันได้	ไม่ได้ป้องกัน
การดักฟังแบบเฉย ๆ (TLS 1.3) การแทรกกลางทาง (proof ผูกกับใบรับรอง) การดัดแปลงข้อมูลระหว่างทาง (SHA-256) การหลุดออกนอกไดเรกทอรีที่แชร์ (Jail) การ replay proof ข้าม connection (nonce)	ผู้ที่อ่านไฟล์ `known_hosts.json` ได้ (เทียบเท่ากุญแจส่วนตัวของ SSH) การวิเคราะห์ปริมาณทราฟฟิก (รู้ขนาดและเวลาของทุกการโอน) การเพิกถอนสิทธิ์รายบุคคล (หมุนรหัสใหม่เพิกถอนทุกคนพร้อมกัน) การรั่วไหลของรหัสจับคู่จากวิธีที่ผู้ใช้แชร์มันเอง

3. Service Model ของ Transport Layer: เลือก TCP เพราะอะไร

คำตอบโดยย่อ

เลือก **TCP** — `AF_INET` + `SOCK_STREAM` พอร์ตเริ่มต้น 5150 ด้วยเหตุผลสองชั้นที่เป็นอิสระจากกัน และแต่ละชั้นเพียงพอในตัวเอง:

(1) ความต้องการของแอปพลิเคชันทั้งหกข้อ ตรงกับการรับประกันของ TCP พอดี การสร้างสิ่งเหล่านี้ขึ้นบน UDP ก็คือการเขียน TCP ขึ้นมาใหม่ให้แย่ลง
(2) TLS 1.3 ทำงานบน TCP และมีในไลบรารีมาตรฐานของ Python ขณะที่ DTLS ซึ่งเป็นส่วนเทียบเท่าบน UDP *ไม่มีให้ใช้เลย* การเลือก UDP จึงเท่ากับต้องเขียนวิทยาการรหัสลับเอง หรือปล่อยโปรแกรมโดยไม่มีการเข้ารหัส

3.1 ความต้องการที่แท้จริงของแอปพลิเคชัน

ก่อนจะเลือก ต้องระบุความต้องการในรูปของ**คุณสมบัติ** ไม่ใช่ในรูปของชื่อโปรโตคอล มิฉะนั้นการเลือกจะกลายเป็นการอ้างธรรมเนียมปฏิบัติ ไม่ใช่การให้เหตุผล

#	ความต้องการ	ทำไมการรับส่งไฟล์จึงต้องการ
R1	**ข้อมูลต้องไม่สูญหาย**	ไฟล์ที่ขาดไปหนึ่งไบต์ ไม่ใช่ไฟล์ที่แย่ลงนิดหน่อย แต่คือไฟล์เสีย
R2	**ส่งถึงตามลำดับ**	ไบต์ที่ 5,000,001 ต้องถูกเขียนหลังไบต์ที่ 5,000,000 ไม่งั้นไฟล์เละ
R3	**ไม่ซ้ำ**	บล็อกที่ส่งซ้ำแล้วถูกเขียนสองครั้ง จะเลื่อนทุกอย่างหลังจากนั้นทั้งหมด
R4	**Flow control**	เครื่องที่อ่านจาก SSD ผลิตข้อมูลได้เร็วกว่าที่แล็ปท็อปซึ่งเขียนลงดิสก์ช้า ๆ จะรับไหว ต้องมีกลไกดันกลับ
R5	**Congestion control**	การอัดข้อมูลเต็มลิงก์ที่ใช้ร่วมกันโดยไม่ยอมถอย เป็นการเอาเปรียบผู้อื่น และในระดับใหญ่ทำให้เกิด congestion collapse
R6	**ความหมายของ connection**	session มีสถานะ — ทักทายแล้ว, พิสูจน์ตัวตนแล้ว, พร้อมทำงาน ทั้งสองฝั่งต้องเห็นตรงกันว่ามี connection อยู่ และจบเมื่อไร
R7	**ความลับและการพิสูจน์ตัวตน**	แบบจำลองภัยคุกคามคือเครื่องอื่นบน Wi-Fi เดียวกัน
R8	**ความถูกต้องแบบ end-to-end**	ไฟล์ที่ลงจอดต้องพิสูจน์ได้ว่าเป็นไฟล์เดียวกับที่ส่งมา

สิ่งที่ไม่ต้องการ: latency ที่มีขอบเขตแน่นอน, throughput ขั้นต่ำ ที่รับประกัน หรือกำหนดเวลาส่งถึงที่ตายตัว — ข้อสังเกตนี้กลายเป็นจุดชี้ขาด (ดู §3.6)

3.2 Service model สองแบบที่มีให้เลือก

คุณสมบัติ	TCP	UDP
การสร้าง connection	มี — three-way handshake และการปิดอย่างชัดเจน	ไม่มี แต่ละ datagram เป็นอิสระ
การรับประกันการส่งถึง	เชื่อถือได้: ACK, timeout, retransmission	Best-effort อาจสูญหาย
ลำดับ	รับประกัน ด้วย sequence number	ไม่มี อาจสลับลำดับ
การตัดข้อมูลซ้ำ	มี	ไม่มี
นามธรรมที่ให้	**สตรีมไบต์ที่เชื่อถือได้** แบบ full-duplex	**ข้อความ (datagram)** ที่ไม่เชื่อถือได้
Flow control	มี — receive window	ไม่มี
Congestion control	มี — AIMD, slow start	ไม่มี
ขนาดเฮดเดอร์	20 ไบต์	8 ไบต์
รับประกันเวลา / throughput	**ไม่มี**	**ไม่มี**
ความปลอดภัย	**ไม่มี**	**ไม่มี**

3.3 การจับคู่ความต้องการกับกลไกของ TCP

```
listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)   # server.py
listener.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
listener.bind((cfg.host, cfg.port))
listener.listen(16)
```

ความต้องการ	กลไกของ TCP ที่ตอบสนอง	ต้นทุนถ้าเลือก UDP
R1 ไม่สูญหาย	Retransmission เมื่อ ACK timeout / duplicate ACK	ต้องเขียน sequence number, ACK, retransmission timer, การประมาณ RTT เอง
R2 ตามลำดับ	Sequence number และการประกอบใหม่ฝั่งรับ	ต้องเขียน reorder buffer เอง
R3 ไม่ซ้ำ	Sequence number	ต้องเขียนการตรวจจับข้อมูลซ้ำเอง
R4 Flow control	Receive window (`rwnd`)	ต้องเขียน windowing และการประกาศ window เอง
R5 Congestion control	AIMD, slow start, fast recovery	ต้องเขียน congestion window และนโยบายถอยเอง
R6 ความหมายของ connection	Three-way handshake และการปิดด้วย FIN	ต้องเขียนการสร้าง session และ timeout เอง

> **ข้อโต้แย้งทั้งหมดรวมอยู่ในคอลัมน์ขวามือ**
>
> รายการในคอลัมน์ขวา **ก็คือ TCP นั่นเอง** การทำ R1–R6 บน UDP คือการเขียน TCP ขึ้นมาใหม่ — และจะเป็นเวอร์ชันที่นักศึกษาคนเดียวเขียนในหนึ่งเทอม ไม่เคยผ่านการตรวจสอบ และรันบนเครือข่ายเดียวเท่านั้น ส่วน TCP ของจริง ถูกโจมตีและตรวจสอบมาตั้งแต่ปี 1981 และอยู่ใน kernel ซึ่งมองเห็น socket buffer และ timer ได้อย่างถูกต้อง

3.4 เหตุผลชี้ขาด: TLS 1.3 ทำงานบน TCP

ยังมีเหตุผลที่สองที่ปิดประเด็นนี้อย่างสิ้นเชิง และเป็นข้อที่ควรเน้น

TLS 1.3 ทำงานบน TCP และไลบรารีมาตรฐานของ Python เปิดให้ใช้ โค้ดของโครงงานนี้จึงไม่ได้เขียนวิทยาการรหัสลับเองแม้แต่บรรทัดเดียว — มันเพียง *ตั้งค่า* สแต็กที่ผ่านการตรวจสอบมาแล้ว

```
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.minimum_version = ssl.TLSVersion.TLSv1_3      # ไม่ยอมรับอะไรที่ต่ำกว่านี้
ctx.load_cert_chain(cert_path, key_path)
```

ส่วนที่เทียบเท่าบน UDP คือ DTLS และโมดูล `ssl` ของ Python ไม่เปิดให้ใช้เลย การเลือก UDP จึงบังคับให้เกิดผลอย่างใดอย่างหนึ่ง

1. **เขียน record layer และ key exchange ขึ้นเอง** ซึ่งเป็นวิธีที่แน่นอนที่สุด ในการสร้างวิทยาการรหัสลับที่พัง และเป็นความผิดพลาดที่วรรณกรรมด้านความปลอดภัย ทั้งหมดเตือนไว้ หรือ
2. **ปล่อยโปรแกรมโดยไม่มีการเข้ารหัส** ซึ่งก็คือ KUFT เวอร์ชันหนึ่ง สิ่งที่เวอร์ชันนี้เกิดมาเพื่อแทนที่

ทั้งสองทางรับไม่ได้ TCP จึงไม่ใช่แค่ตัวเลือกที่ดีกว่า แต่**เป็นตัวเลือกเดียวที่ทำให้โครงงานนี้มีการเข้ารหัสได้เลย**

3.5 ช่องว่างใน service model ของ TCP ที่ต้องเติมเอง

TCP ไม่ได้แก้ทุกอย่างให้ Service model ของมันไม่สมบูรณ์สำหรับแอปพลิเคชันนี้ อยู่ห้าข้อ และแต่ละช่องว่างถูกเติมอย่างจงใจที่ชั้น Application

ช่องว่าง	ทำไมจึงเป็นปัญหา	สิ่งที่ KUFT เติมเข้าไป
สตรีมไบต์ไม่มีขอบเขตข้อความ	`send` กับ `recv` ไม่สอดคล้องกัน หนึ่ง write อาจมาถึงเป็นสามการอ่าน หรือสาม write อาจมาถึงเป็นครั้งเดียว ฝ่ายรับจึงบอกไม่ได้ว่าข้อความจบตรงไหน	Framing ที่ชัดเจน: ส่วนหัวจบด้วยบรรทัดว่าง บวก `content-length` ที่บังคับมี (§4.2)
TCP checksum อ่อนแอและครอบคลุมแค่ทีละ hop	เป็นผลรวมแบบ one's-complement 16 บิต ออกแบบมาจับสัญญาณรบกวน คำนวณต่อ segment และ*ถูกคำนวณใหม่โดยอุปกรณ์กลางทางทุกตัว* การแก้ไขของผู้โจมตีเชิงรุกจึงผ่านฉลุย	SHA-256 แบบ end-to-end ทั้งต่อช่วงข้อมูลและต่อไฟล์ทั้งไฟล์ (§4.11)
ไม่มีความลับและไม่มีการพิสูจน์ตัวตน	TCP ส่ง plaintext ทั้งหมด และไม่รู้ว่าปลายทางเป็นใคร	TLS 1.3 บวกกลไกจับคู่ที่ผูก proof เข้ากับใบรับรอง (§4.10)
รู้แค่ว่า "เชื่อมต่อแล้ว" ไม่รู้ว่า "พิสูจน์ตัวตนแล้ว"	TCP connection ที่เปิดอยู่ไม่ได้แปลว่าคู่สนทนามีสิทธิ์ทำอะไรก็ได้	State machine ระดับแอปพลิเคชัน: ทุกคำสั่งยกเว้น `HELLO` และ `AUTH` ตอบ `401` จนกว่า proof จะผ่าน (§4.12)
connection ที่ขาดทำให้เสียการโอนทั้งครั้ง	TCP กู้คืน *segment* ไม่ใช่ *ไฟล์* ถ้า connection ตาย ดาวน์โหลด 4 GB ที่เดินมาถึง 90% ต้องเริ่มใหม่จากศูนย์	เก็บไฟล์ `.part` ที่ยังไม่ครบไว้ แล้วเริ่มต่อด้วยเฮดเดอร์ `range` (§4.11)

3.6 สิ่งที่ TCP ไม่รับประกัน แต่เราไม่ต้องการ

TCP จงใจ**ไม่รับประกัน throughput ขั้นต่ำและไม่รับประกันขอบเขต latency** สำหรับแอปพลิเคชันจำนวนมากนั่นคือข้อจำกัดที่แท้จริง แต่ที่นี่มันไม่เสียหายอะไรเลย และเหตุผลควรพูดให้ตรง

> **ในการรับส่งไฟล์ ไบต์ที่มาช้ามีค่าเท่ากับไบต์ที่มาตรงเวลาทุกประการ**
>
> ไม่มีอะไรถูกนำไปใช้จนกว่าไฟล์ทั้งไฟล์จะมาถึงและ hash ผ่าน การโอนที่หยุดชะงัก ไปสองวินาทีแล้วไปต่อ ให้ผลลัพธ์เหมือนกันทุกไบต์ ดังนั้นการที่ TCP ยอมแลก latency เพื่อความน่าเชื่อถือ — รอ retransmission หรือลด congestion window ลงครึ่งหนึ่ง — เป็นการแลกที่โครงงานนี้ *ยินดีเสมอ*

3.7 กรณีที่ UDP จะเป็นคำตอบที่ถูกต้อง

UDP ถูกต้องเมื่อ **datagram ที่มาช้าถือว่าไร้ค่า ไม่ใช่แค่ล่าช้า**

กรณี	เหตุผล
เสียงและวิดีโอสด	แพ็กเก็ตที่มาช้า 400 ms ไม่เหลือช่วงเวลาให้เล่นแล้ว การกลบช่องว่างดีกว่าการรอ
สถานะเกม	ข้อมูลอัปเดตชิ้นถัดไปแทนที่ชิ้นก่อนหน้า การส่งสถานะเก่าซ้ำมีแต่โทษ
DNS	การทำ handshake แพงกว่าตัวคำถามเสียอีก
Multicast / Broadcast	TCP ทำไม่ได้เลยโดยธรรมชาติ เพราะเป็นโปรโตคอลแบบจุดต่อจุด

ไม่มีข้อไหนตรงกับการย้ายไฟล์ อย่างไรก็ตาม มีจุดเดียวในโครงงานนี้ ที่ UDP จะเป็นคำตอบที่ถูกต้องจริง ๆ คือ **mDNS service discovery** เพื่อไม่ต้องพิมพ์หมายเลข IP — เพราะเป็นงานแบบ best-effort, multicast และลองใหม่ที่ชั้นแอปพลิเคชันได้ แต่ยังไม่ได้ทำในเวอร์ชันนี้

4. การออกแบบ Application-Layer Protocol

หัวข้อนี้คือข้อกำหนดของโปรโตคอล KUFT/2.0 อย่างละเอียด ครอบคลุมรูปแบบข้อความ ชุดคำสั่ง เฮดเดอร์ รหัสสถานะ และ **request/response messages ทุกคู่** ที่ใช้สื่อสารระหว่าง Client และ Server

4.1 ข้อจำกัดที่การออกแบบต้องทำให้ได้

1. **Framing ต้องไม่ขึ้นกับเนื้อหาของ body** — ไฟล์ไบนารีมีทุกไบต์ที่เป็นไปได้ รวมทั้ง CRLF และ NUL การแบ่งข้อความต้องไม่ถูกรบกวนจากเนื้อหาไฟล์
2. **ต้องสตรีมได้** — ห้ามโหลดไฟล์ทั้งไฟล์เข้าหน่วยความจำ
3. **ต้องรองรับชื่อไฟล์ทุกแบบ** — เว้นวรรค วงเล็บ อักษรไทย อิโมจิ
4. **ต้องอ่านได้จาก packet capture** — เพื่อให้ดีบักและตรวจสอบได้
5. **ต้องกันการใช้หน่วยความจำไม่จำกัด** — คู่สนทนาที่ไม่หวังดีต้องทำให้ เซิร์ฟเวอร์จองหน่วยความจำไม่จำกัดไม่ได้

4.2 รูปแบบข้อความ

ข้อความของ KUFT คือ **ส่วนหัวที่เป็นข้อความ บรรทัดว่างหนึ่งบรรทัด แล้วตามด้วย body ไบนารีที่นับจำนวนไว้แล้ว**

```
message   = head CRLF [ body ]
head      = start-line CRLF *( header CRLF )
header    = field-name ":" OWS field-value
```

- `start-line` เป็น US-ASCII
- ชื่อเฮดเดอร์ไม่สนใจตัวพิมพ์เล็กใหญ่ และถูกแปลงเป็นตัวพิมพ์เล็กเมื่อรับ
- ค่าเฮดเดอร์เป็น UTF-8 และ**ต้องไม่มี CR หรือ LF** ตัว encoder จะปฏิเสธที่จะ*เข้ารหัส*ค่าที่มีอักขระเหล่านี้ ซึ่งเป็นสิ่งที่ปิดช่องโหว่ header injection
- ส่วนหัวจบที่บรรทัดว่างแรก
- มี body **ก็ต่อเมื่อ** มีเฮดเดอร์ `content-length` ที่ไม่เป็นศูนย์ body มีขนาดเท่านั้นพอดี และไม่ถูกตีความใด ๆ จึงมี CRLF, NUL หรือไบต์อะไรก็ได้โดยไม่ต้อง escape

> **ทำไมจึงเลือกรูปทรงแบบ HTTP**
>
> นี่คือรูปแบบของ HTTP และการหยิบยืมนี้เป็นความตั้งใจ: มันคือ framing ชั้นแอปพลิเคชันที่ประสบความสำเร็จที่สุดเท่าที่เคยมีการใช้งานมา อ่านออกได้ทันที และคุณสมบัติของมันเป็นที่เข้าใจกันดี สิ่งที่ **ไม่ได้** หยิบยืมมา คือความซับซ้อนที่ HTTP สะสมมา — ไม่มี chunked encoding, ไม่มี content negotiation, ไม่มี cookie, ไม่มี caching
>
> **ทางเลือกที่พิจารณาแล้วไม่เลือก:** *binary TLV* กะทัดรัดดี แต่มองไม่เห็นใน packet capture และไบต์ที่ประหยัดได้ไม่มีความหมายเลยเมื่อเทียบกับ body ขนาดหลายกิกะไบต์ · *JSON* บรรทุกข้อมูลไบนารีดิบไม่ได้ถ้าไม่ base64 ซึ่งเพิ่มขนาด 33% ในทุก ๆ ไฟล์

4.3 Request line และ Status line

Request line

```
request-line = method SP target SP "KUFT/2.0"
```

`target` ถูก **percent-encode**: ทุกอ็อกเท็ตที่อยู่นอก `A-Z a-z 0-9 / * - . _ ~` ถูกเขียนเป็น `%XX` บนการเข้ารหัส UTF-8 ของมัน สิ่งนี้รับประกันว่า request line มี **สาม token คั่นด้วยช่องว่างเสมอ** ไม่ว่าไฟล์จะชื่ออะไร และทำให้ชื่อ Unicode ทุกแบบโอนได้ครบถ้วน

```
GET /Quarterly%20Report%20(final).pdf KUFT/2.0
PUT /%E0%B8%A3%E0%B8%B2%E0%B8%A2%E0%B8%87%E0%B8%B2%E0%B8%99.txt KUFT/2.0
```

Status line

```
status-line = "KUFT/2.0" SP status-code SP reason-phrase
```

4.4 การตัดสินใจสี่ข้อเรื่อง framing

การตัดสินใจ	ผลที่ตามมา
1. ส่วนหัวเป็น ASCII และจบที่บรรทัดว่างแรก	Parser **ไม่เคยตรวจสอบ body เลย** ไฟล์จึงมี CRLF, NUL หรืออะไรก็ได้ โดยไม่ต้อง escape และไม่มีต้นทุน ถ้าใช้ end marker แทน จะต้องสแกนหา marker ใน body ซึ่งแปลว่าต้อง escape marker ภายใน body ซึ่งแปลว่า framing ขึ้นกับเนื้อหา นั่นคือกฎที่ยืนยันว่าจะไม่ยอมละเมิด
2. `content-length` บังคับมีเสมอเมื่อมี body	ฝ่ายรับรู้แน่นอนว่าต้องอ่านกี่ไบต์ และไม่ต้องเดาว่าข้อความจบตรงไหน ทำให้ส่งข้อความติด ๆ กันบน connection เดียวได้อย่างปลอดภัย และเป็นสิ่งที่ทำให้ สตรีมได้ — อ่านตามจำนวนที่รู้แล้ว ทีละบล็อก 256 KiB ตรงเข้าไฟล์ปลายทาง คำนวณ hash ไปด้วยระหว่างทาง **ไฟล์ 40 GB ใช้หน่วยความจำเท่ากับไฟล์ 4 KB**
3. Target ถูก percent-encode	รับประกันว่า request line มีสาม token เสมอ ถ้าไม่มีมัน `GET Quarterly Report final.pdf KUFT/2.0` จะถูกแยกเป็นห้า token และ parse ไม่ผ่าน — ซึ่งเป็นจุดที่เวอร์ชันหนึ่งพังจริง ๆ
4. ทุกความยาวที่คู่สนทนามีอิทธิพลต่อได้ มีเพดานตายตัว	คู่สนทนาที่ส่งบรรทัดเฮดเดอร์โดยไม่มีตัวจบบรรทัด จะทำให้เซิร์ฟเวอร์ จองหน่วยความจำไม่จำกัดไม่ได้ (ดู §4.5)

> **กฎเล็ก ๆ ที่ปิดช่องโหว่จริง**
>
> **ตัว encoder ปฏิเสธที่จะส่งค่าเฮดเดอร์ที่มี CR หรือ LF** ถ้าไม่มีข้อนี้ ชื่อไฟล์ที่มีตัวขึ้นบรรทัดใหม่อยู่ข้างในสามารถแทรกเฮดเดอร์ปลอม หรือแทรกแม้กระทั่ง status line ปลอม เข้าไปในคำตอบได้ การปฏิเสธตั้งแต่ตอน encode หมายความว่าไม่มีผู้เรียกใช้รายไหนเปิดช่องโหว่นี้ได้เลย

4.5 ขีดจำกัดที่บังคับใช้

ขีดจำกัด	ค่า	บังคับด้วย
บรรทัดควบคุม	8192 อ็อกเท็ต	`400`
จำนวนเฮดเดอร์ต่อข้อความ	64	`400`
Control body (listing, proof)	4 MiB	`413`
Upload body	`--max-upload` ของเซิร์ฟเวอร์ (ค่าเริ่มต้น 8 GiB)	`413`
ขนาดบล็อกในการโอน	256 KiB	—

4.6 ชุดคำสั่ง (Methods)

เก้าคำสั่งสำหรับการทำงาน บวก `QUIT` เลือกให้แต่ละคำสั่งจับคู่กับ ปฏิบัติการบนระบบไฟล์หนึ่งอย่างพอดี และไม่มีคำสั่งใดครอบคลุมคำสั่งอื่น

Method	Target	Body	จุดประสงค์
`HELLO`	`*`	—	ตกลงความสามารถ รับ salt และ server nonce
`AUTH`	`*`	—	พิสูจน์ว่ารู้รหัสจับคู่ และรับ proof จากเซิร์ฟเวอร์
`LIST`	ไดเรกทอรี	—	รายการไฟล์ในรูปแถวคั่นด้วย tab
`STAT`	ไฟล์	—	ขนาด เวลาแก้ไข checksum โดยไม่โอนข้อมูล
`GET`	ไฟล์	—	ดาวน์โหลด; `range: N` ขอเฉพาะส่วนท้าย
`PUT`	ไฟล์	payload	อัปโหลด; ปรับพฤติกรรมด้วย `overwrite`, `range`
`DELETE`	path	—	ลบไฟล์หรือไดเรกทอรีว่าง
`MKDIR`	ไดเรกทอรี	—	สร้างไดเรกทอรี
`MOVE`	path	—	เปลี่ยนชื่อ/ย้าย; เฮดเดอร์ `destination` ระบุปลายทาง
`QUIT`	`*`	—	ปิดการเชื่อมต่ออย่างสุภาพ

สิ่งที่จงใจไม่มี: ไม่มี `COPY` (ใช้ `GET` + `PUT` ฝั่งไคลเอนต์), ไม่มี `APPEND` (ใช้ `PUT` พร้อม `range`), ไม่มี `CHMOD` (สิทธิ์ไม่อยู่ในแบบจำลอง) และไม่มี `DELETE` แบบวนลึก (ไดเรกทอรีที่ไม่ว่างจะได้ `409`)

`STAT` มีคุณค่าตรงที่ทำให้การปฏิเสธ `PUT` มีต้นทุนต่ำ: ไคลเอนต์ถามก่อน อัปโหลดขนาด 40 GB จึงไม่ถูกส่งขึ้นลิงก์ช้า ๆ เพื่อมาโดนปฏิเสธด้วย `409` ตอนจบ

4.7 เฮดเดอร์ทั้งหมด

เฮดเดอร์	ทิศทาง	ความหมาย
`content-length`	ทั้งสองทาง	ขนาด body เป็นอ็อกเท็ต บังคับมีเมื่อมี body
`content-type`	Server → Client	ชนิดของ body เช่น `application/octet-stream`
`detail`	Server → Client	คำอธิบายที่มนุษย์อ่านได้ มีในทุกคำตอบ
`salt`	Server → Client	Salt สำหรับ PBKDF2 (base64) ใน `HELLO`
`nonce`	ทั้งสองทาง	ค่าสุ่ม 32 อ็อกเท็ต (base64) ใหม่ทุก connection
`proof`	ทั้งสองทาง	HMAC-SHA256 (base64) ที่ผูกกับใบรับรอง
`auth`	Server → Client	วิธีพิสูจน์ตัวตนที่รองรับ ปัจจุบันคือ `pairing-code`
`server`	Server → Client	ชื่อและเวอร์ชันของเซิร์ฟเวอร์
`methods`	Server → Client	รายการคำสั่งที่รองรับ คั่นด้วยจุลภาค
`tls`	Server → Client	เวอร์ชัน TLS ที่เจรจาได้จริง
`read-only`	Server → Client	`true` ถ้าเซิร์ฟเวอร์ปฏิเสธการเขียนทั้งหมด
`max-upload`	Server → Client	ขนาดอัปโหลดสูงสุดที่ยอมรับ
`checksum`	ทั้งสองทาง	`sha256=<hex>` ของช่วงข้อมูลที่ส่ง
`whole-checksum`	Server → Client	`sha256=<hex>` ของไฟล์ทั้งไฟล์
`size`	Server → Client	ขนาดไฟล์ทั้งไฟล์เป็นอ็อกเท็ต
`offset`	Server → Client	ตำแหน่งเริ่มต้นของ body ภายในไฟล์
`range`	Client → Server	ขอข้อมูลตั้งแต่อ็อกเท็ตที่ N เป็นต้นไป
`overwrite`	Client → Server	`true` เพื่ออนุญาตให้ `PUT` ทับไฟล์เดิม
`destination`	Client → Server	path ปลายทางของ `MOVE` (percent-encoded)
`kind`	Server → Client	`file` หรือ `dir` ในคำตอบของ `STAT`
`modified`	Server → Client	เวลาแก้ไขล่าสุดแบบ ISO 8601 UTC
`count`	Server → Client	จำนวนรายการในคำตอบของ `LIST`
`path`	Server → Client	path ที่ resolve แล้ว แสดงเทียบกับรากที่แชร์
`retry-after`	Server → Client	จำนวนวินาทีที่ต้องรอ มากับ `429`
`connection`	Server → Client	`close` เมื่อเซิร์ฟเวอร์จะปิด connection

4.8 รหัสสถานะ (Status codes)

ใช้รหัสสามหลักตามคลาสของ HTTP เพื่อให้ *คลาส* มีความหมาย แม้ผู้รับจะไม่รู้จักรหัสเฉพาะตัวนั้น — `2xx` สำเร็จ, `4xx` เป็นความผิดของไคลเอนต์, `5xx` เป็นความผิดของเซิร์ฟเวอร์

Success

รหัส	วลี	เมื่อไร
200	OK	คำขอสำเร็จ; `PUT` ทับไฟล์ที่มีอยู่แล้ว
201	CREATED	`PUT` หรือ `MKDIR` สร้างของใหม่
204	NO CONTENT	`DELETE` สำเร็จ
206	PARTIAL CONTENT	`GET` ที่มี `range`; body เป็นส่วนท้ายของไฟล์

Client errors

รหัส	วลี	เมื่อไร
400	BAD REQUEST	Framing ที่ parse ไม่ได้ — **ปิด connection หลังจากนั้น**
401	UNAUTHORIZED	ยังไม่พิสูจน์ตัวตน หรือ proof ตรวจไม่ผ่าน
403	FORBIDDEN	path หลุดออกนอกไดเรกทอรีที่แชร์, เซิร์ฟเวอร์เป็น read-only, หรือเป้าหมายคือรากที่แชร์
404	NOT FOUND	ไม่มีไฟล์หรือไดเรกทอรีนั้น
405	METHOD NOT ALLOWED	คำสั่งที่ไม่รู้จัก
409	CONFLICT	เป้าหมายมีอยู่แล้วและไม่ได้ขอทับ; หรือไดเรกทอรีไม่ว่าง
413	PAYLOAD TOO LARGE	Body เกินขีดจำกัดที่ตั้งไว้
416	RANGE NOT SATISFIABLE	`range` อยู่นอกขอบเขตไฟล์ หรือ offset ที่เซิร์ฟเวอร์จับคู่ไม่ได้
422	**CHECKSUM MISMATCH**	ข้อมูลที่รับมาไม่ตรงกับ checksum ที่ประกาศไว้
429	TOO MANY REQUESTS	เกิน rate limit; `retry-after` บอกจำนวนวินาที

Server errors

รหัส	วลี	เมื่อไร
500	INTERNAL ERROR	ความล้มเหลวที่ไม่คาดคิดขณะจัดการคำขอที่ถูกต้อง
503	SERVICE UNAVAILABLE	ถึงขีด `--max-clients`; ส่งก่อน session จะเริ่ม
505	VERSION NOT SUPPORTED	คำขอประกาศเวอร์ชันอื่นที่ไม่ใช่ `KUFT/2.0`

422 CHECKSUM MISMATCH — รหัสเดียวที่เป็นของ KUFT โดยแท้

ความหมายคือ *คำขอถูกต้องตามรูปแบบ และข้อมูลมาถึงครบตามที่ชั้น transport รับรู้ แต่ payload ไม่ตรงกับ checksum ที่ผู้ส่งประกาศไว้* ไม่มีรหัส HTTP ใดหมายความแบบนั้น และนี่คือความล้มเหลวที่สำคัญที่สุดที่โปรโตคอลนี้ต้องรายงานได้ เพราะมันคือจุดที่ ความถูกต้องแบบ end-to-end (R8) ถูกบังคับใช้จริง

4.9 Request และ Response messages ทุกคำสั่ง

หัวข้อนี้แสดงข้อความจริงที่วิ่งบนสาย ทุกตัวอย่างเป็นสิ่งที่โปรแกรมส่งจริง สัญลักษณ์ `C -> S` คือไคลเอนต์ส่งไปเซิร์ฟเวอร์ และ `S -> C` คือเซิร์ฟเวอร์ตอบกลับ

HELLO — เปิด session และรับค่าสำหรับการจับคู่

```
C -> S   HELLO * KUFT/2.0
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         server:      kuft-server/2.0 (win32)
         methods:     HELLO,AUTH,LIST,STAT,GET,PUT,DELETE,MKDIR,MOVE,QUIT
         tls:         TLSv1.3
         auth:        pairing-code
         salt:        3q2+7wAAAAB/f39/f39/fw==
         nonce:       kzD1nP0aQ2h5c2VydmVyIG5vbmNlIGhlcmUgMzJC
         read-only:   false
         max-upload:  8589934592
         detail:      send AUTH with your pairing proof
         <บรรทัดว่าง>
```

AUTH — พิสูจน์ตัวตนสองทาง

```
C -> S   AUTH * KUFT/2.0
         nonce:  Y2xpZW50IG5vbmNlIDMyIG9jdGV0cyBoZXJlLi4u
         proof:  9Gm1kZ0vQ0hLbXlDbGllbnRQcm9vZkJhc2U2NA==
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         proof:   XxQ2c2VydmVyUHJvb2ZCYXNlNjRFbmNvZGVkLi4=
         detail:  authenticated
         <บรรทัดว่าง>

--- กรณีรหัสผิด ---
S -> C   KUFT/2.0 401 UNAUTHORIZED
         detail:  pairing proof did not verify
         <บรรทัดว่าง>
         (ตอบหลังหน่วงเวลา 1 วินาที และ session ยังไม่ถูกปิด
          ผู้ใช้จึงพิมพ์รหัสใหม่ได้)
```

LIST — รายการไฟล์

Body เป็นแถวคั่นด้วย tab รูปแบบ `kind ⇥ size ⇥ modified ⇥ name` ไดเรกทอรีมี size เป็น `-`

```
C -> S   LIST /lecture KUFT/2.0
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         content-type:    text/tab-separated-values; charset=utf-8
         content-length:  147
         count:           3
         path:            /lecture
         detail:          3 entries
         <บรรทัดว่าง>
         DIR     -       2026-08-24T09:12:03Z    slides
         FILE    4096    2026-08-24T09:14:55Z    checksums.md
         FILE    21870   2026-08-25T01:02:41Z    notes.md
```

STAT — ข้อมูลไฟล์โดยไม่โอนข้อมูล

```
C -> S   STAT /report.pdf KUFT/2.0
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         kind:      file
         size:      8213
         modified:  2026-08-25T14:31:07Z
         checksum:  sha256=1f3a9c07be5d21e4a8f0c6b93d47e15a0982f4cbb17d6e3a9042f8c1d5b7e604
         detail:    metadata for /report.pdf
         <บรรทัดว่าง>
```

GET — ดาวน์โหลดทั้งไฟล์

```
C -> S   GET /report.pdf KUFT/2.0
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         content-type:     application/octet-stream
         content-length:   8213
         size:             8213
         offset:           0
         checksum:         sha256=1f3a9c07be5d21e4a8f0c6b93d47e15a...
         whole-checksum:   sha256=1f3a9c07be5d21e4a8f0c6b93d47e15a...
         detail:           sending /report.pdf
         <บรรทัดว่าง>
         <ข้อมูล 8213 ไบต์พอดี ไม่ถูกตีความใด ๆ>
```

เมื่อ `offset` เป็น 0 ค่า `checksum` และ `whole-checksum` จะเท่ากันเสมอ

GET พร้อม range — ดาวน์โหลดต่อจากที่ค้างไว้

```
C -> S   GET /big.bin KUFT/2.0
         range: 1048576
         <บรรทัดว่าง>

S -> C   KUFT/2.0 206 PARTIAL CONTENT
         content-type:     application/octet-stream
         content-length:   3145728
         size:             4194304
         offset:           1048576
         checksum:         sha256=<ของอ็อกเท็ตที่ 1048576 เป็นต้นไป>
         whole-checksum:   sha256=<ของไฟล์ทั้ง 4194304 ไบต์>
         detail:           sending /big.bin
         <บรรทัดว่าง>
         <ข้อมูล 3145728 ไบต์>

--- กรณี range เกินขนาดไฟล์ ---
S -> C   KUFT/2.0 416 RANGE NOT SATISFIABLE
         detail:  range 9999999 outside 0..4194304
         <บรรทัดว่าง>
```

PUT — อัปโหลด

```
C -> S   PUT /outbox/report.md KUFT/2.0
         content-length:  5120
         checksum:        sha256=7b1e44d0c9a2f83e5610bd47ca03f9182e...
         <บรรทัดว่าง>
         <ข้อมูล 5120 ไบต์>

S -> C   KUFT/2.0 201 CREATED
         detail:  stored /outbox/report.md (5120 octets)
         <บรรทัดว่าง>

--- ถ้าไฟล์มีอยู่แล้วและไม่ได้ขอทับ ---
S -> C   KUFT/2.0 409 CONFLICT
         detail:  /outbox/report.md exists; resend with overwrite
         <บรรทัดว่าง>
         (ตอบ *ก่อน* อ่าน body จึงไม่เปลืองแบนด์วิดท์)

--- ถ้าขอทับด้วย overwrite: true และสำเร็จ ---
S -> C   KUFT/2.0 200 OK
         detail:  replaced /outbox/report.md (5120 octets)
         <บรรทัดว่าง>

--- ถ้าข้อมูลที่รับมาไม่ตรงกับ checksum ---
S -> C   KUFT/2.0 422 CHECKSUM MISMATCH
         detail:  expected sha256=7b1e44d0..., got sha256=c04ab918...
         <บรรทัดว่าง>
         (ไฟล์ชั่วคราวถูกลบทิ้ง ไฟล์เดิมไม่ถูกแตะต้อง)

--- ถ้า body เกิน --max-upload ---
S -> C   KUFT/2.0 413 PAYLOAD TOO LARGE
         detail:  body 9663676416 exceeds limit 8589934592
         <บรรทัดว่าง>
```

DELETE, MKDIR, MOVE

```
C -> S   DELETE /old.txt KUFT/2.0
S -> C   KUFT/2.0 204 NO CONTENT
         detail:  removed /old.txt

C -> S   MKDIR /archive KUFT/2.0
S -> C   KUFT/2.0 201 CREATED
         detail:  created /archive

C -> S   MOVE /report.pdf KUFT/2.0
         destination: /archive/report.pdf
S -> C   KUFT/2.0 200 OK
         detail:  /report.pdf -> /archive/report.pdf

--- ไดเรกทอรีที่ไม่ว่างจะลบไม่ได้ ---
S -> C   KUFT/2.0 409 CONFLICT
         detail:  /lecture is not empty

--- เซิร์ฟเวอร์ที่เปิด --read-only ปฏิเสธทุกคำสั่งที่เขียน ---
S -> C   KUFT/2.0 403 FORBIDDEN
         detail:  server is read-only
```

QUIT — ปิดอย่างสุภาพ

```
C -> S   QUIT * KUFT/2.0
         <บรรทัดว่าง>

S -> C   KUFT/2.0 200 OK
         connection:  close
         detail:      goodbye
         <บรรทัดว่าง>
```

ตัวอย่างข้อผิดพลาดอื่น

```
--- คำสั่งที่ต้องพิสูจน์ตัวตนก่อน แต่ยังไม่ได้ AUTH ---
C -> S   GET /secret.txt KUFT/2.0
S -> C   KUFT/2.0 401 UNAUTHORIZED
         detail:  authenticate first

--- path ที่พยายามหลุดออกนอกไดเรกทอรีที่แชร์ ---
C -> S   GET /../../etc/passwd KUFT/2.0
S -> C   KUFT/2.0 403 FORBIDDEN
         detail:  path escapes the share
         (เป็น 403 ไม่ใช่ 404 เสมอ คำตอบจึงไม่เปิดเผยว่า path
          ข้างนอกนั้นมีอยู่จริงหรือไม่)

--- เวอร์ชันโปรโตคอลไม่ตรงกัน ---
C -> S   GET /a.txt KUFT/1.0
S -> C   KUFT/2.0 505 VERSION NOT SUPPORTED
         detail:  this server speaks KUFT/2.0

--- เกิน rate limit ---
S -> C   KUFT/2.0 429 TOO MANY REQUESTS
         retry-after:  7
         detail:       rate limit exceeded

--- เกินจำนวนไคลเอนต์สูงสุด (ส่งภายใน TLS ก่อน session เริ่ม) ---
S -> C   KUFT/2.0 503 SERVICE UNAVAILABLE
         connection:  close
         detail:      server at capacity (8 clients)
```

4.10 กลไกจับคู่ (Pairing exchange)

นี่คือส่วนที่ทำให้การเข้ารหัสมีความหมาย จึงกำหนดไว้อย่างแม่นยำ

> **ปัญหา**
>
> TLS ให้ช่องทางที่เป็นความลับ — แต่ใบรับรองของเซิร์ฟเวอร์เป็นแบบ **self-signed** บนเครือข่ายที่มีแค่แล็ปท็อปสองเครื่อง ไม่มี certificate authority ให้อ้างอิง ใบรับรองเพียงลำพังจึง *ไม่พิสูจน์อะไรเลย* ผู้โจมตีสร้างขึ้นเองได้ง่ายพอ ๆ กับเรา และ **การเข้ารหัสไปหาคู่สนทนา ที่ไม่รู้ว่าเป็นใคร แทบไม่มีค่า** — มันป้องกันคนที่นั่งฟังเฉย ๆ ได้ แต่ไม่ป้องกันคนที่แทรกอยู่ตรงกลางเลย

> **แนวคิดหลัก**
>
> **ผูก proof ของการพิสูจน์ตัวตนเข้ากับใบรับรองที่ถูกนำเสนอจริง**

4.10.1 ค่าที่เกี่ยวข้อง

สัญลักษณ์	นิยาม
`code`	อักขระ 12 ตัวจาก RFC 4648 base32 — **60 บิต** ไม่สนใจตัวพิมพ์ ละเว้นขีด และอ่าน `0 1 8 9` เป็น `O I B G` (ชุดอักขระจึงไม่มีเลขที่สับสนกับตัวอักษรเวลาอ่านออกเสียง)
`salt`	16 อ็อกเท็ตสุ่ม สร้างครั้งเดียวต่อเซิร์ฟเวอร์และเก็บไว้
`K`	`PBKDF2-HMAC-SHA256(code, salt, 200000, 32)`
`FP`	`SHA-256(DER(server certificate))` — 32 อ็อกเท็ต
`Nc` , `Ns`	32 อ็อกเท็ตสุ่ม **ใหม่ทุก connection**

4.10.2 การสร้าง proof

โดยให้ `L(x)` คือความยาวของ `x` แบบ big-endian 4 อ็อกเท็ต

```
proof(label) = HMAC-SHA256( K,
                 L(label) || label ||
                 L(FP)    || FP    ||
                 L(Nc)    || Nc    ||
                 L(Ns)    || Ns    )

client proof: label = "KUFT/2.0 client-proof"
server proof: label = "KUFT/2.0 server-proof"
```

4.10.3 ลำดับการแลกเปลี่ยน

```
                    TLS 1.3 handshake เสร็จสิ้น
                              |
C -> S   HELLO * KUFT/2.0
                              |
S -> C   KUFT/2.0 200 OK
         salt:  <base64 salt>
         nonce: <base64 Ns>
         auth:  pairing-code
                              |
         ผู้ใช้พิมพ์รหัสจับคู่ที่อ่านจากหน้าจอเซิร์ฟเวอร์
         ไคลเอนต์คำนวณ K แล้วสร้าง proof บน FP ที่ตนเห็น
                              |
C -> S   AUTH * KUFT/2.0
         nonce: <base64 Nc>
         proof: <base64 proof("KUFT/2.0 client-proof")>
                              |
         เซิร์ฟเวอร์ตรวจสอบด้วยการเปรียบเทียบแบบเวลาคงที่
                              |
S -> C   KUFT/2.0 200 OK
         proof: <base64 proof("KUFT/2.0 server-proof")>
                              |
         ไคลเอนต์ตรวจ proof ของเซิร์ฟเวอร์ จึงรู้ว่าอีกฝั่งถือรหัสเดียวกัน
                              |
                    [ authenticated ]
```

4.10.4 การโจมตีแบบดักกลางทางล้มเหลวอย่างไร

สมมติผู้โจมตีแทรกอยู่ตรงกลาง ปิด TLS ทั้งสองด้านด้วยใบรับรองของตัวเอง ซึ่งมีลายนิ้วมือ `FP'`

1. ไคลเอนต์คำนวณ proof บน `FP'` เพราะนั่นคือใบรับรองที่มันถูกนำเสนอ
2. ผู้โจมตีส่ง proof นั้นต่อไปยังเซิร์ฟเวอร์จริง — มันคำนวณใหม่ไม่ได้ เพราะการคำนวณใหม่ต้องใช้ `K`
3. เซิร์ฟเวอร์ตรวจ proof เทียบกับ `FP` ซึ่งเป็นลายนิ้วมือของตัวเอง
4. `FP ≠ FP'` **MAC จึงไม่ตรงกัน และเซิร์ฟเวอร์ตอบ** `401`

ผู้โจมตีซ่อมสถานการณ์นี้ไม่ได้ถ้าไม่มี `K` และหา `K` ไม่ได้ ถ้าไม่มีรหัส — **และรหัสนั้นไม่เคยวิ่งผ่านเครือข่ายในรูปแบบใดเลย** ไม่ได้เข้ารหัสส่ง ไม่ได้ hash ส่ง ไม่ได้ส่งเลย ผู้ใช้อ่านมันจากหน้าจอ proof ที่ถูกดักได้จึงให้เพียงการโจมตีแบบ offline ต่อ 60 บิต ที่มี PBKDF2 200,000 รอบขวางอยู่

4.10.5 องค์ประกอบแต่ละตัวกันการโจมตีอะไร

องค์ประกอบ	การโจมตีที่ถูกปิด
ลายนิ้วมือใบรับรองอยู่ใน MAC	การแทรกกลางทาง (machine-in-the-middle)
nonce ใหม่จากทั้งสองฝั่ง	proof ที่ถูกดักไว้ใช้ได้กับ connection เดียวและไม่มีวันใช้ได้อีก (replay)
label สองอันที่ต่างกัน	proof ของไคลเอนต์ถูกส่งย้อนกลับเป็น proof ของเซิร์ฟเวอร์ไม่ได้ (reflection)
length prefix ในทุกฟิลด์	ทำให้การเข้ารหัส input เป็นแบบ injective — ไม่มี input สองชุดที่ต่างกันให้ไบต์ชุดเดียวกัน
PBKDF2 200,000 รอบ	การเดารหัสแบบ offline ทำได้ช้าลงมหาศาล
การเปรียบเทียบแบบเวลาคงที่	Timing side channel
base32 ที่ไม่มี 0 1 8 9	ความผิดพลาดของมนุษย์เวลาอ่านรหัสออกเสียง (O กับ 0 สับสนกันไม่ได้)

4.10.6 การปักหมุด (Pinning)

เมื่อการแลกเปลี่ยนสำเร็จครั้งแรก ไคลเอนต์เก็บ `FP` และ `K` ไว้คู่กับ `host:port` ใน `~/.kuft/known_hosts.json` การเชื่อมต่อครั้งต่อ ๆ ไปจึงข้ามการใส่รหัสไปเลยและใช้ `K` เดิม

ถ้า `FP` ที่เก็บไว้ไม่ตรงกับที่ถูกนำเสนอ ไคลเอนต์ **ยกเลิกการเชื่อมต่อทันที ไม่มี prompt ให้กดยืนยัน**

> นั่นเป็นการตัดสินใจออกแบบโดยเจตนา ไม่ใช่การลืมใส่ ลายนิ้วมือที่เปลี่ยนหมายถึง อย่างใดอย่างหนึ่ง: เซิร์ฟเวอร์ถูกสร้างใหม่ หรือคุณกำลังถูกดักอยู่ — **และกล่องข้อความให้กดยืนยัน คือวิธีที่คนกดผ่านกรณีหลังพอดี** การกู้คืนจึงต้องใช้คำสั่งแยกต่างหากอย่างตั้งใจ คือ `kuft.py forget <host>` ซึ่งบังคับให้ผู้ใช้ต้องหยุดแล้วคิด

4.11 ความหมายของการโอนย้าย

4.11.1 ดาวน์โหลด

`GET` ตอบกลับด้วย `size`, `checksum` (ของช่วงข้อมูลที่ส่ง), `whole-checksum` (ของไฟล์ทั้งไฟล์) และ body

1. ไคลเอนต์เขียนข้อมูลลงไฟล์ `<ชื่อ>.part` และคำนวณ hash **ไปพร้อมกับการเขียน** จึงไม่ต้องอ่านซ้ำ
2. เทียบกับ `checksum` ของช่วงข้อมูล
3. เมื่อสำเร็จ hash ไฟล์ที่ประกอบเสร็จแล้วเทียบกับ `whole-checksum`
4. **ทั้งสองต้องผ่าน** จึงจะเปลี่ยนชื่อ `.part` เป็นชื่อจริง
5. ถ้าล้มเหลว ไฟล์ `.part` จะยังอยู่ — ซึ่งเป็นสิ่งที่ทำให้ ครั้งถัดไปทำต่อได้ด้วย `range` แทนที่จะเริ่มใหม่

4.11.2 อัปโหลด — เป็นแบบ atomic

1. เซิร์ฟเวอร์เขียน body ลงไฟล์ชั่วคราว **ในไดเรกทอรีปลายทาง**
2. คำนวณ hash และเทียบกับ `checksum` ที่ไคลเอนต์ประกาศไว้
3. *จึงค่อย* `os.replace()` เข้าตำแหน่งจริง

เพราะไฟล์ชั่วคราวอยู่ไดเรกทอรีเดียวกับปลายทาง การเปลี่ยนชื่อจึงอยู่ในระบบไฟล์ เดียวกันและเป็น atomic และการเปลี่ยนชื่อเกิด*หลัง* checksum ตรงเท่านั้น ดังนั้น **การอัปโหลดที่ขาดตอนหรือถูกดัดแปลง ไม่มีทางทับไฟล์ดี ๆ ได้เลย** ไฟล์เวอร์ชันเดิมรอดโดยไม่ถูกแตะต้อง

หากมีเฮดเดอร์ `range: N` เซิร์ฟเวอร์จะต่อท้ายจากตำแหน่ง `N` โดยมีเงื่อนไขว่าขนาดไฟล์ปัจจุบันต้องเท่ากับ `N` พอดี มิฉะนั้นตอบ `416`

4.12 State machine ของ session

```
                TLS 1.3 handshake
                       |
                       v
                [ connected ]
                       |  HELLO
                       v
                 [ greeted ]
                       |  AUTH พร้อม proof ที่ถูกต้อง
                       v
               [ authenticated ] <--+
                       |            | LIST STAT GET PUT
                       |  QUIT      | DELETE MKDIR MOVE
                       v            |
                  [ closed ]  ------+
```

- คำสั่งอื่นใดนอกจาก `HELLO` หรือ `AUTH` ในสถานะ `connected` หรือ `greeted` → `401`
- `AUTH` ก่อน `HELLO` → `400` เพราะยังไม่มี `Ns` ให้ผูก proof เข้าด้วย
- ข้อผิดพลาด framing ระดับ `400` **ปิด connection** เพราะเมื่อสตรีมหลุดจากการซิงก์แล้ว ไม่มีวิธีหาขอบเขตข้อความถัดไปได้อีก
- `QUIT` ถูกตอบด้วย `200` พร้อม `connection: close`

4.13 การกักขอบเขต (Jail)

ทุก target ถูก resolve เทียบกับรากที่แชร์ แล้วจึงตรวจว่าเป็นรากนั้นเอง หรือเป็นลูกหลานของมัน การ resolve จะยุบ `..` และ **ตาม symlink ก่อนการตรวจ** ดังนั้น symlink ที่ชี้ออกนอกไดเรกทอรีที่แชร์ จึงถูกปฏิเสธเช่นเดียวกับ `../` ที่เขียนตรง ๆ

ความล้มเหลวตอบเป็น `403` เสมอ ไม่ใช่ `404` และคำตอบไม่เปิดเผยว่า path ข้างนอกนั้นมีอยู่จริงหรือไม่

5. สรุปคำตอบ

คำถาม	คำตอบ
เป็นโปรแกรมอะไร ใช้ทำอะไร	โปรแกรมรับส่งไฟล์แบบ client/server ที่ย้ายไฟล์โดยตรงระหว่างสองเครื่อง บนเครือข่ายเดียวกัน โดยเข้ารหัสและตรวจสอบความถูกต้อง ไม่ผ่านบุคคลที่สาม ไม่ต้องมีบัญชี และไม่ต้องตั้งค่าอะไรนอกจากรหัสจับคู่หนึ่งรหัส
มีคุณลักษณะอย่างไร	Client/server, session แบบมีสถานะและอายุยาว, ข้อมูลสองทาง, ขนาดไฟล์ไม่จำกัด, **ทนต่อการสูญหายเป็นศูนย์** แต่ **ทนต่อความหน่วงได้สูงมาก**, ต้องการความปลอดภัยเต็มรูปแบบ, ผู้ใช้พร้อมกันน้อย (8 ราย), หนึ่ง thread ต่อหนึ่ง connection และใช้หน่วยความจำคงที่ด้วยการสตรีมทีละ 256 KiB
ใช้ service model ใด เพราะอะไร	**TCP** — connection-oriented, เชื่อถือได้, เรียงลำดับ, ไม่ซ้ำ, เป็นสตรีมไบต์ พร้อม flow control และ congestion control ความต้องการ R1–R6 ตรงกับการรับประกันของ TCP พอดี การสร้างสิ่งเหล่านี้บน UDP ก็คือการเขียน TCP ใหม่ให้แย่ลง **และเหตุผลชี้ขาดคือ TLS 1.3 มีให้ใช้บน TCP ในไลบรารีมาตรฐาน ขณะที่ DTLS ไม่มี** การเลือก UDP จึงเท่ากับต้องเขียนวิทยาการรหัสลับเอง หรือไม่มีการเข้ารหัสเลย ส่วนสิ่งที่ TCP ขาดถูกเติมที่ชั้นแอปพลิเคชันทุกจุด และสิ่งเดียวที่มันขาดจริง ๆ คือการไม่รับประกันเรื่องเวลา ซึ่งไม่เสียหายอะไรเลย เพราะในการรับส่งไฟล์ ไบต์ที่มาช้ามีค่าเท่ากับไบต์ที่มาตรงเวลาทุกประการ
ออกแบบโปรโตคอลอย่างไร	ส่วนหัวเป็น US-ASCII ที่จบด้วยบรรทัดว่าง บวก `content-length` ที่บังคับมี — framing จึงไม่ขึ้นกับเนื้อหาของ body; target ถูก percent-encode — request line จึงมีสาม token เสมอ; เก้าคำสั่งที่แต่ละอันตรงกับปฏิบัติการบน ระบบไฟล์หนึ่งอย่าง; รหัสสถานะตามคลาสของ HTTP บวก `422 CHECKSUM MISMATCH`; และ proof ของการจับคู่ที่ผูกกับช่องทาง โดยใส่ลายนิ้วมือใบรับรองเข้าไปใน HMAC ซึ่งเป็นสิ่งที่ทำให้การเข้ารหัสมีความหมาย